



Universidad
Indoamérica

UNIT 5 · FUNDAMENTALS OF OPERATING SYSTEMS

Pipes and process composition

Universidad Tecnológica Indoamérica Lab TD4

How do two isolated processes communicate?

Byte streams

Pipes (this unit), FIFOs (unit 3), sockets (unit 8)

Notifications

Signals (unit 6)

Shared memory

`mmap` and shared memory (unit 6), threads (unit 7)

The pipe is the simplest and most used one: every `|` in the shell is one.

A pipe: `pipe(fds)`

- Creates a kernel buffer and two descriptors: `fds[0]` to read and `fds[1]` to write
- One direction, a byte stream with no messages and no name
- Only processes that inherit the descriptors share it: **it is created before** `fork`
- Each process **closes the end it does not use**

C tds/td4/mypipe.c

```
exit_if(pipe(fds) == -1, "pipe");
pid_t pid = fork();
exit_if(pid == -1, "fork");

if (pid == 0) {
    exit_if(close(fds[1]) == -1, "close"); /* ch.
    char buffer[READ_MAX];
    ssize_t n = read(fds[0], buffer, READ_MAX); /* on
    exit_if(n == -1, "read");
    exit_if(write_all(STDOUT_FILENO, buffer, (size_t)n)
    fprintf(stderr, "I am the child, I read %zd bytes an
    exit_if(close(fds[0]) == -1, "close");
    return EXIT_SUCCESS;
```

The rules of a pipe

Read with the pipe empty	blocks... or returns 0 if there is no writer left
--------------------------	--

Write with the pipe full	blocks until someone reads
--------------------------	----------------------------

Write with no reader at all	SIGPIPE signal (or EPIPE if ignored)
-----------------------------	--

Write $\leq$ PIPE_BUF (4096 B)	atomic: it does not mix with others
---------------------------------------	-------------------------------------

Default capacity on the course machine:

Real output

```
$ python3 -c 'import os, fcntl; r, w = os.pipe();  
print(fcntl.fcntl(w, fcntl.F_GETPIPE_SZ), "bytes")'  
65536 bytes
```

10, 300, 10,000 messages: what happens?

timeline (model, μs)

parent

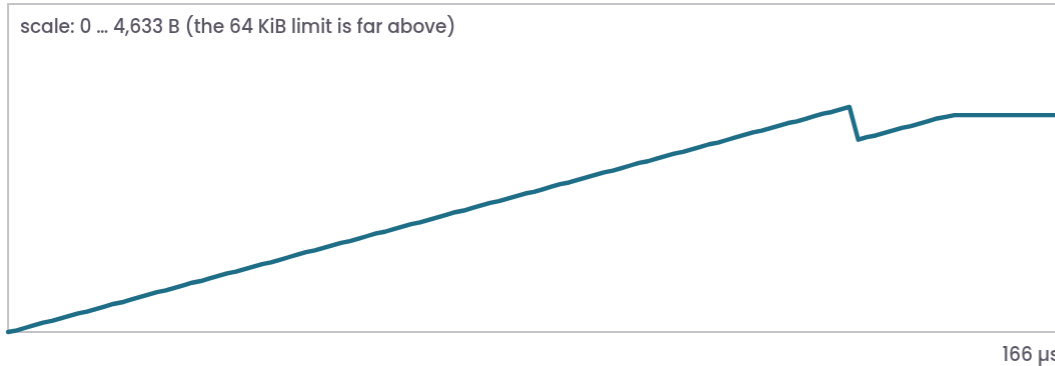


child

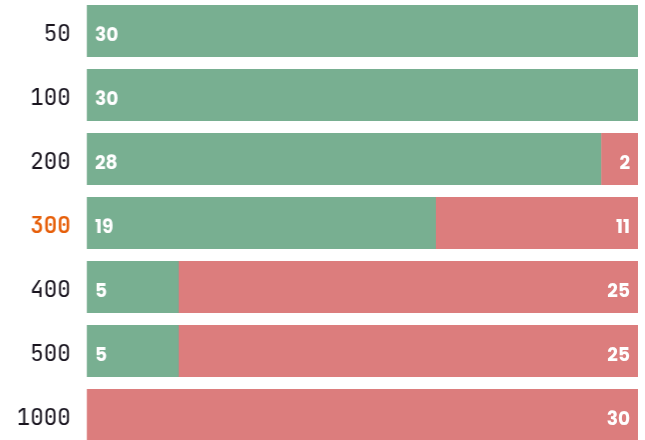


bytes in the pipe

scale: 0 ... 4,633 B (the 64 KiB limit is far above)



real measurement: 30 runs per size



green: status 0 · red: 141 (SIGPIPE)
messages

Closing or not closing `fds[0]` in the parent

Real output

```
$ ./mypipe 10000 > /dev/null; echo "exit status of mypipe: $?"  
I am the child, I read 500 bytes and I leave  
exit status of mypipe: 141
```

Real output

```
$ timeout 3 ./mypipe_bug 10000 > /dev/null; echo "exit status: $? (124 = killed by timeout: it was blocked)"  
I am the child, I read 500 bytes and I leave  
exit status: 124 (124 = killed by timeout: it was blocked)
```

With the end closed, the parent gets SIGPIPE (141 = 128 + 13) when the child leaves. Without closing it, the parent is its own "reader": it fills the 64 KiB and blocks forever (124 = killed by `timeout`). When run with `&`, the shell reports the exit status.

Plugging in someone else's program: `dup2` + `exec`

C tds/td4/upper.c

```
/* parent */
exit_if(close(fds[0]) == -1, "close");
exit_if(dup2(fds[1], STDOUT_FILENO) == -1, "dup2");
exit_if(close(fds[1]) == -1, "close");          /* 1 already closed */
execlp("ps", "ps", (char *)NULL);              /* "ps" to be executed */
exit_if(1, "execlp ps");
```

Real output

```
$ ./upper | head -4
  PID TTY          TIME CMD
 6706 ?            00:00:25 UVICORN
 9771 ?            00:00:03 SYSTEMD
 9801 ?            00:00:00 (SD-PAM)
```

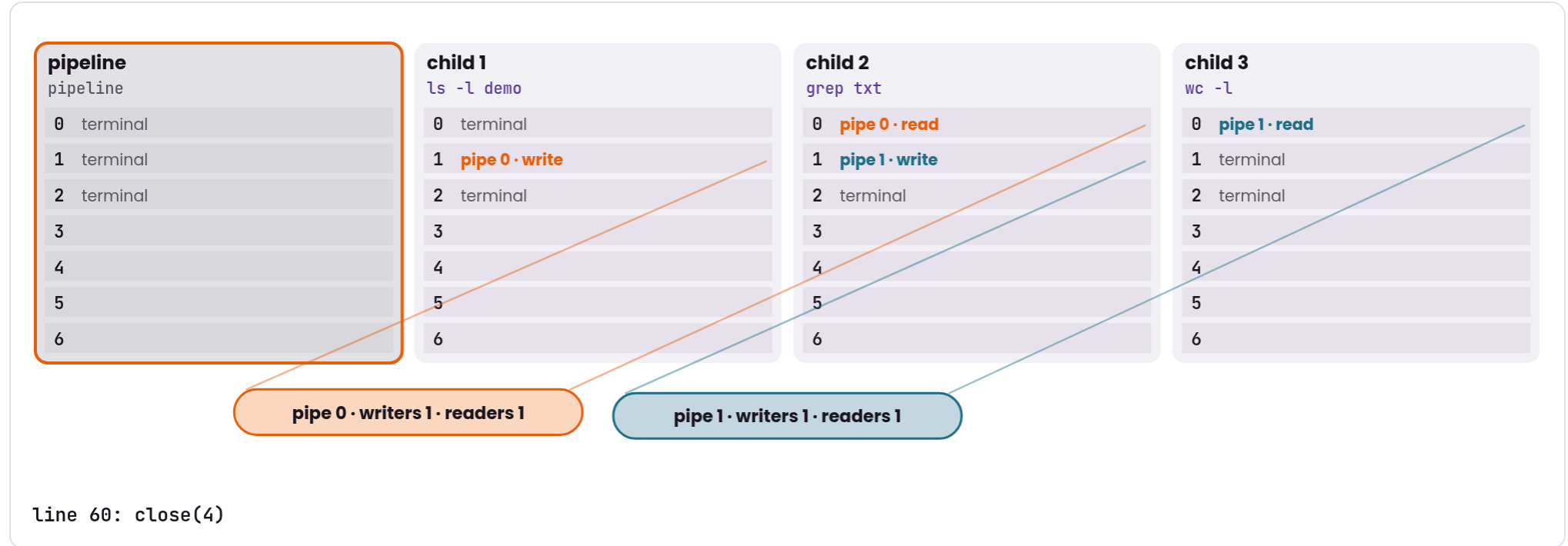
`ps` knows nothing about pipes: it writes to its descriptor 1, which `dup2` connected to the pipe. The child reads until end of file and converts to upper case.

What the shell does with

```
ls -l | tr -d '[:blank:]'
```

1. `pipe(fds)`
2. `fork` → child 1: `dup2(fds[1], 1)`, closes both, `exec("ls")`
3. `fork` → child 2: `dup2(fds[0], 0)`, closes both, `exec("tr")`
4. The shell closes both of its ends and waits for both children

The descriptor tables, step by step



N commands and the pipeline's status

C tds/td4/pipeline_args.c

```
if (pid == 0) {
    if (in != STDIN_FILENO) {
        exit_if(dup2(in, STDIN_FILENO) == -1, "dup2");
        exit_if(close(in) == -1, "close");
    }
    if (fds[1] != -1) {
        exit_if(dup2(fds[1], STDOUT_FILENO) == -1,
            exit_if(close(fds[0]) == -1 || close(fds[1]
    }
    execvp(commands[i][0], commands[i]);
    perror(commands[i][0]);
    _exit(127);
}
if (in != STDIN_FILENO)
    exit_if(close(in) == -1, "close");
if (fds[1] != -1) {
    exit_if(close(fds[1]) == -1, "close");
    in = fds[0];
}
```

Real output

```
$ ./pipeline_args ls -l demo -- grep txt -- wc -l
2
```

Real output

```
$ ./pipeline_args ls -- sh -c 'exit 3'; echo "status: $?"
status: 3
```

Real output

```
$ ./pipeline_args ls -- sh -c 'kill -TERM $$'
pipeline: last command killed by SIGTERM
[exit status 1]
```

Like the shell, it returns the status of the **last** command (`set -o pipefail` changes that rule in bash).

Beyond pipe

pipe2

`O_CLOEXEC` keeps the pipe from leaking into programs launched with `exec` ; `O_NONBLOCK` to avoid blocking.

splice

Moves data between a pipe and a file or socket without copying it into the process.

Capacity

`fcntl(F_SETPIPE_SZ)` enlarges it up to `/proc/sys/fs/pipe-max-size`.

Solved TD4 + AI challenge

- `mypipe` with 10, 300 and 10,000 messages; with and without the bug
- `upper` : `ps` in upper case
- `pipeline_args ls -l -- grep txt -- wc -l`

Bash

```
make -C tds && tds/tests/run.sh td4
```



Universidad
Indoamérica

TAKE-AWAYS

Three questions

1. When does a `read` on a pipe return 0?
2. Why does `mypipe 10000` end with 141 and `mypipe_bug` never ends?
3. In `a | b | c`, how many `pipe`, `fork` and `close` calls does the shell make?